SP.TOP: ENCRYPTION THEORY - COMP438

First Semester 2025-2026

Project Two

LFSR Stream Cipher

With Nonlinear Keystream Generation

Doaa Assi – 1220504

Course instructor: Mohammad Alkhanafseh

Data: 28/1/2025

Abstract

To improve the security of a basic Linear Feedback Shift Register (LFSR), this project uses a Nonlinear Feedback Shift Register (NLFSR). Filtering functions produce multiple bits per round in a nonlinear way, and combining functions mix outputs from two rounds to create a more secure keystream. This approach shows how adding nonlinearity makes a simple linear system stronger and harder to predict.

Contents

Theory 1

Explanation of the implementation 2

1. Generate key for baseline: 2

2. Encryption process: 3

3. Decryption process: 4

Enhancement 6

1. Generate key for enhancement: 7

Results and Evaluation 8

1. Results a Baseline LFSR 8

2. Results an Enhancement LFSR (NLFSR) 8

3. Evaluation 9

Weaknesses and Limitations 10

Analysis of resistance to basic attacks 10

Attack Scenarios 11

1. Keystream Reuse Attack 11

2. Known Plaintext Attack 12

Figure

Figure 1: example of how LFSR 1
Figure 2: code generate keystream for baseline LFSR 2
Figure 3: code generate seed 3
Figure 4: code encryption message 4
Figure 5: read cipher text and seed 4
Figure 6: seed extraction 5
Figure 7: code decryption message 5
Figure 8: code generate keystream for enhancement LFSR 7
Figure 9: results a baseline LFSR 8
Figure 10: results an enhancement LFSR 8
Figure 11: result Keystream Reuse Attack 11
Figure 12: result known plaintext attack 12

Table

Table 1: example of how LFSR 1
Table 2: Evaluation 9

Theory

In LFSR Stream Cipher, the first step is to define a seed. Then, each round generates a new key using the values from the seed and LFSR, and all the values generated from each round are combined to obtain a keystream. This keystream is used to encrypt the original text using XOR; that is, each bit of the text is combined with its corresponding bit from the keystream. This method is fast and easy to implement, but it is vulnerable if the same seed is used more than once or if the keystream becomes predictable.

Example:

Seed: 101001

B6 = B6 XOR B1

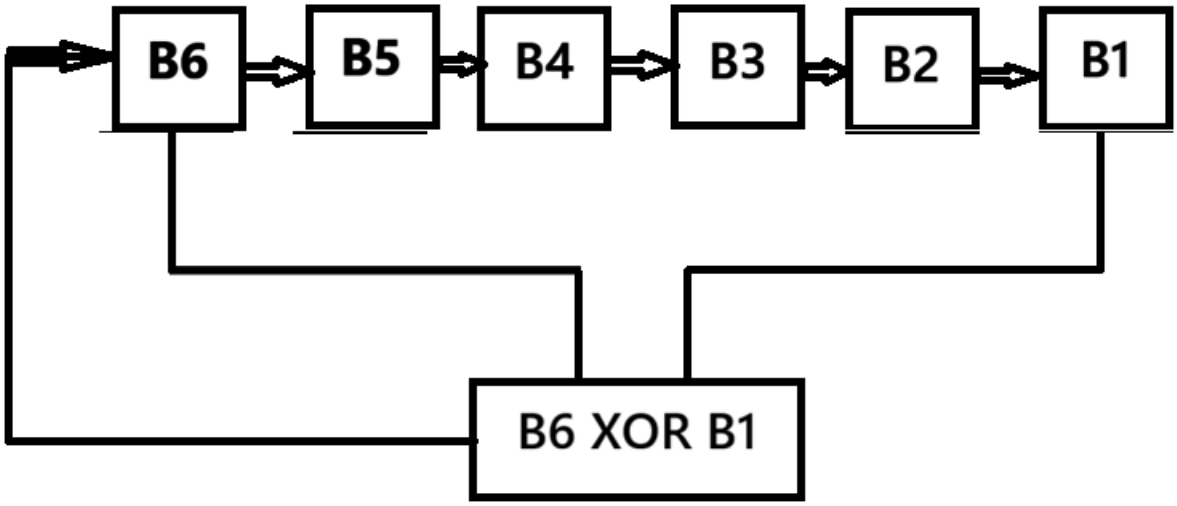

Figure 1: example of how LFSR

Table 1: example of how LFSR

B6		B5		B4		B3		B2		B1	B6 = B6 XOR B1
1	↘	0	↘	1	↘	0	↘	0	↘	1	1 XOR 1 = 0
0	↘	1	↘	0	↘	1	↘	0	↘	0	0 XOR 0 = 0
0	↘	0	↘	1	↘	0	↘	0	↘	0	0 XOR 0 = 0
0	↘	0	↘	0	↘	1	↘	0	↘	0	0 XOR 0 = 0
0	↘	0	↘	0	↘	0	↘	1	↘	0	0 XOR 0 = 0
0	↘	0	↘	0	↘	0	↘	0	↘	1	0 XOR 1 = 1
1	↘	0	↘	0	↘	0	↘	0	↘	0	1 XOR 0 = 1

1	↘	1	↘	0	↘	0	↘	0	↘	0	1 XOR 0 = 1

⇨ Keystream: 1010010101000010000001000000100000011000001100000…..

Explanation of the implementation

This section will explain the baseline code used in the project. As for the enhancement code, it generally follows the same encryption and decryption steps, but the main difference lies in the keystream generation method. Therefore, the details of the enhancement and the improved key generation mechanism will be explained separately in a later section.

1. Generate key for baseline:

In this design, the seed length was chosen to be 6 bits, so in each round, the LFSR generates 6 new bits that are used as part of the keystream. The feedback function is an XOR operation between the sixth bit and the first bit: ` (b6 ⊕b1) `, with the register being moved from left to right. This function was chosen after studying the concept of Primitive Polynomial, as using a suitable function helps achieve the longest possible keystream generation time before iterating, which is approximately `(2n − 1)`, thus increasing the strength and randomness of the encryption. Keystream generation is stopped when the desired length of plaintext is reached, to avoid generating a large number of unnecessary bits. This ensures that only a suitable keystream length is used, making the process more efficient without compromising encryption.

```
n = 6
def generate_key_for_baseline(seed, binary_message):
    listKey = []
    key = [str(bit) for bit in seed.copy()] # b6, b5, b4, b3, b2, b1 => key[0], key[1], key[2], key[3], key[4], key[5]
    keystream = ''.join(key)    "keystream": Unknown word.
    listKey.append(''.join(key))
    for _ in range((pow(2, n))-1):
        b1 = key[5]
        b6 = key[0]
        # Shift bits to the right
        for i in range(n-1, 0, -1):
            key[i] = key[i-1]
        key[0] = str(int(b6) ^ int(b1)) # b6 = b1 xor b6
        # if  ''.join(key) in listKey:
        #     break
        keystream += ''.join(key)    "keystream": Unknown word.
        listKey.append(''.join(key))
        if len(keystream) >= len(binary_message):    "keystream": Unknown word.
            break
    return keystream    "keystream": Unknown word.
```

Figure 2: code generate keystream for baseline LFSR

Initially, a condition was used to stop keystream generation once it began to repeat, but this approach later proved entirely illogical. The reason is that if the keystream length were shorter than the plaintext length, the encryption would have to reuse the same key directly and systematically, potentially reducing security. Therefore, the LFSR was allowed to continue generating until the desired message length was reached, even if internal repetition occurred, because this repetition wasn't obvious or systematic to an attacker, thus somewhat mitigating the risk of overt keystream reuse.

2. Encryption process:

The system receives the original message that the sender wants to encrypt, then a random seed value is generated to run the LFSR and generate the keystream. The first bit from the left is always set to 1 to avoid reaching a dead state, a condition in which the entire register becomes zero and therefore no new bits are generated.

```
class LFSR_encryption:    "LFSR": Unknown word.
    # n = 6
    def __init__(self, message, type):
        self.message = message
        self.type = type
        self.seed = [1] + [random.randint(0, 1) for _ in range(n - 1)] # ensure seed is not all 0s: default first bit is 1
        self.binary_message = ''.join(format(ord(char), '08b') for char in message)
```

Figure 3: code generate seed

In the encryption process, the keystream is first generated by calling the LFSR's key generation function. If the keystream is shorter than the message length, it is reused until it is sufficient to cover the entire plaintext. The plaintext is then converted to a binary representation, and an XOR operation is performed between the bits of the keystream and the bits of the message. An XOR operation is performed on each of the 8 bits once, and the resulting character is converted to form the cipher text.

To transmit the seed value between the sender and receiver, it is included as part of the cipher text itself, where the seed is represented as the first character of the cipher text. The entire cipher text is then saved in a file, which represents the transmission process between the two parties.

```python
def encryption_message(self):
    keystream = ''    "keystream": Unknown word.
    if self.type == "baseline":
        keystream = generate_key_for_baseline(self.seed, self.binary_message)    "keystream": Unknown word.
    else:
        keystream = generate_key_for_enhancement(self.seed, self.binary_message)    "keystream": Unknown word.
    if not keystream:    "keystream": Unknown word.
        print("Keystream generation failed.")    "Keystream": Unknown word.
        return
    while len(keystream) < len(self.binary_message):    "keystream": Unknown word.
        keystream += keystream    "keystream": Unknown word.
    ciphertext = ''    "ciphertext": Unknown word.
    encrypted_message = ''
    for i in range(0, len(self.binary_message), 8):
        byte_msg = int(self.binary_message[i:i+8], 2)
        byte_key = int(keystream[i:i+8], 2)    "keystream": Unknown word.
        encrypted_byte = byte_msg ^ byte_key
        encrypted_message += format(encrypted_byte, '08b')
        ciphertext += chr(encrypted_byte)    "ciphertext": Unknown word.
    seed_char = chr(int(''.join(str(bit) for bit in self.seed), 2))
    text_to_save = seed_char + ciphertext    "ciphertext": Unknown word.
    if self.type == "baseline":
        with open(r"C:\Users\LTC\Desktop\FileProject\encryption_project2\encrypted_baseline.txt", "w", encoding="utf-8") as file:
            # file.write(''.join(str(bit) for bit in self.seed) + "\n")
            # file.write(str(ciphertext))    "ciphertext": Unknown word.
            file.write(str(text_to_save))
            file.close()
    else:
        with open(r"C:\Users\LTC\Desktop\FileProject\encryption_project2\encrypted_enhancement.txt", "w", encoding="utf-8") as file:
            file.write(str(text_to_save))
            file.close()
    print("ciphertext as binary:", encrypted_message)    "ciphertext": Unknown word.
    print("ciphertext:", ciphertext)    "ciphertext": Unknown word.
    print("Encryption complete. Encrypted message saved to file")
    return
```

Figure 4: code encryption message

3. Decryption process:

The program reads the file, takes the first letter as the seed, and treats the rest of the file as cipher text.

```python
print("Baseline decryption selected")
with open(r"C:\Users\LTC\Desktop\FileProject\encryption_project2\encrypted_baseline.txt", "r", encoding="utf-8") as file:
    content = file.read()
seed = content[0]
ciphertext = content[1:]    "ciphertext": Unknown word.
decryptor = LFSR_decryption(seed, ciphertext, type="baseline")    "decryptor": Unknown word.
message = decryptor.decryption_message()    "decryptor": Unknown word.
```

Figure 5: read cipher text and seed

To extract the original seed, it is first converted into an 8-bit binary representation, and then the last n bits from the right are taken to ensure that the same seed used during the encryption process is obtained. Also, the cipher text is converted into a binary representation so that we can perform mathematical operations on it.

```python
class LFSR_decryption:    "LFSR": Unknown word.
    def __init__(self, seed, ciphertext, type):    "ciphertext": Unknown word.
        seed_bin = format(ord(seed), '08b')
        self.seed = [int(b) for b in seed_bin[-n:]]
        print("Seed is binary:", self.seed)
        self.ciphertext = ciphertext    "ciphertext": Unknown word.
        self.type = type
        self.binary_message = ''.join(format(ord(char), '08b') for char in ciphertext)
```

Figure 6: seed extraction

In the keystream generation process, the same generation method used during encryption is employed to ensure that the same key used to encrypt the original text is obtained. Each 8 bits are decoded once by an XOR process with 8 bits of the keystream, and the result produces the character, and thus the characters are extracted one after the other.

```python
def decryption_message(self):
    plaintext = ''
    if self.type == "baseline":
        keystream = generate_key_for_baseline(self.seed, self.binary_message)
    else:
        keystream = generate_key_for_enhancement(self.seed, self.binary_message)
    for i in range(0, len(self.binary_message), 8):
        byte_key = int(keystream[i:i+8], 2)    "keystream": Unknown word.
        byte_cipher = int(self.binary_message[i:i+8], 2)
        decrypted_byte = byte_cipher ^ byte_key
        plaintext += chr(decrypted_byte)
    return plaintext
```

Figure 7: code decryption message

Enhancement

The traditional LFSR system is enhanced by introducing nonlinearity using NLFSR to increase security. The system relies on filtering functions to extract bits nonlinearly from the state of the register in each round. A combining function is then used to merge the outputs from multiple rounds to produce a more random keystream. This approach makes the system more difficult to analyze and predict compared to linear systems, and contributes to improving the overall security characteristics of the generator.

1. Filtering functions:

   Instead of using all the bits generated from each round, only a limited number of bits are extracted using filtering functions. Each resulting bit is generated by applying nonlinear operations such as AND, OR, and XOR to the original state bits of the NLFSR. This approach introduces nonlinearity into the system and makes the relationship between inputs and outputs more complex, thus increasing the difficulty of keystream analysis or prediction.

2. Combining function:

   After applying filtering functions, a combining function is used to merge the outputs of multiple rounds. For example, the bits extracted from two consecutive rounds are taken and combined using nonlinear operations such as XOR between corresponding or overlapping bits. The goal of this process is to increase the correlation between rounds and make the resulting keystream more random.

This system relies on increasing security by introducing nonlinearity into keystream generation. Using NLFSR instead of LFSR reduces the system's vulnerability to linear attacks because the relationship between bits becomes indirect and difficult to analyze. Furthermore, applying filtering functions to extract a limited number of bits and then combining the outputs of multiple rounds using a combining function increases randomness and obscures the internal structure of the register. This approach makes keystream prediction or recovery of the register's original state more difficult, thus enhancing the system's security characteristics.

COMBINING AND FILTERING FUNCTIONS IN THE FRAMEWORK OF NONLINEAR FEEDBACK SHIFT REGISTER

1. Generate key for enhancement:

During the Enhancement phase, a nonlinear optimization was applied to the bit generation process within the LFSR. The value of the new bit was modified using the following equation:

b6 = b1 XOR b6 XOR (b4 AND b3) XOR ((b5 OR b2) AND (b6 OR b1))

This modification adds greater complexity compared to traditional linear generation, which helps improve the randomness of the keystream and increases its resistance to attacks. In addition, nonlinear filtering was implemented using nonlinear equations based on the six bit values, where only four bits are extracted in each round. The output of two consecutive rounds is then combined using further operations to obtain six new bits, which are added as part of the final keystream used in encryption.

```python
def generate_key_for_enhancement(seed, binary_message):
    filtering = []
    listKey = []
    key = [str(bit) for bit in seed.copy()] # b6, b5, b4, b3, b2, b1 => key[0], key[1], key[2], key[3], key[4], key[5]
    keystream = ''.join(key)    "keystream": Unknown word.
    listKey.append(''.join(key))
    for _ in range((pow(2, n))-1):
        f1 = int(key[4]) &  int(key[2]) | int(key[5]) # b2 and b4 or b1
        f2 = int(key[3]) & int(key[1]) | int(key[4]) # b3 and b5 or b2
        f3 = int(key[2]) & int(key[0]) | int(key[3]) # b4 and b6 or b3
        f4 = f3 ^ f2 ^ f1
        new_filter = str(f4) + str(f3) + str(f2) + str(f1)
        filtering.append(new_filter)
        b1 = key[5]
        b6 = key[0]
        b2 = key[4]
        b3 = key[3]
        b4 = key[2]
        b5 = key[1]
        # Shift bits to the right
        for i in range(n-1, 0, -1):
            key[i] = key[i-1]
        # b6 = b1 xor b6 xor (b4 and b3) xor ((b5 or b2) and (b6 or b1))
        new_bit = (int(b6) ^ int(b1)) ^ (int(b4) & int(b3)) ^ ((int(b5) | int(b2)) & (int(b6) | int(b1)))
        key[0] = str(new_bit)
        if  ''.join(key) in listKey:
            break
        keystream += ''.join(key)    "keystream": Unknown word.
        listKey.append(''.join(key))
```

```python
        for i in range(len(filtering)):
            f1 = filtering[i]
            f2 = filtering[(i+1) % len(filtering)]
            c1 = int(f1[0]) ^ int(f2[0])
            c2 = int(f1[1]) ^ int(f2[1])
            c3 = int(f1[2]) ^ int(f2[2])
            c4 = int(f1[3]) ^ int(f2[3])
            c5 = int(f1[0]) ^ int(f2[2])
            c6 = int(f1[1]) ^ int(f2[3])
            combined = str(c1) + str(c2) + str(c3) + str(c4) + str(c5) + str(c6)
            keystream += combined    "keystream": Unknown word.
            if len(keystream) >= len(binary_message):    "keystream": Unknown word.
                break
    return keystream    "keystream": Unknown word.
```

Figure 8: code generate keystream for enhancement LFSR

Results and Evaluation

During encryption, the algorithm was executed successfully without any issues, producing the correct cipher text. In decryption, the receiver was able to retrieve the original text in its entirety, demonstrating that they were able to extract the correct seed value and regenerate the same keystream used in encryption. This confirms that the system functions correctly in both the baseline and enhanced states without any errors during retrieval.

1. Results a Baseline LFSR

```
Baseline encryption selected
Message to encrypt: Hi
seed: [1, 0, 1, 0, 1, 0]
ciphertext as binary: 1110001100111111
ciphertext: ã?
seed + ciphertext: *ã?
Encryption complete. Encrypted message saved to file
Baseline decryption selected
Seed is binary: [1, 0, 1, 0, 1, 0]
Seed: *
Ciphertext: ã?
Decrypted message: Hi
```

Figure 9: results a baseline LFSR

2. Results an Enhancement LFSR (NLFSR)

```
Enhancement encryption selected
Message to encrypt: Ok
seed: [1, 0, 1, 0, 1, 1]
ciphertext as binary: 1110001101101011
ciphertext: ãk
seed + ciphertext: +ãk
Encryption complete. Encrypted message saved to file
Enhancement decryption selected
Seed is binary: [1, 0, 1, 0, 1, 1]
Seed: +
Ciphertext: ãk
Decrypted message: Ok
```

Figure 10: results an enhancement LFSR

The resulting cipher text typically appears as gibberish or strange symbols. This is because the encryption process relies on bit-level XOR, producing binary values that may not fall within the range of readable ASCII characters. Therefore, the cipher text appears as unreadable symbols,

which is normal and common in LFSR Stream Cipher algorithms. However, when decryption is performed using the correct keystream, the original text is recovered without any problems.

3. Evaluation

Table 2: Evaluation

	Baseline LFSR	Enhancement LFSR	Comment
Randomness "Balanced"	#0s: 186 #1s: 192 Total: 378	#0s: 56 #1s: 52 Total: 108	The number of 0s and 1s is nearly equal, indicating a good balance of bits within the keystream. This balance is important because it makes the encryption more random and reduces key prediction, thus increasing encryption strength and resistance to attacks.
Period Length "Long enough"	# of rounds: 63	# of rounds: 17	In the baseline keystream, the number of rounds was high because we used a linear LFSR based on a primitive polynomial, which provided the longest possible cycle before the record started repeating. In the enhancement keystream, we added some non-linear complexity, so the polynomial is no longer primitive, and the number of rounds is reduced. However, this increased encryption security.
Avalanche Effect "Sensitive"	#of variable bits:192 Avalanche%: 50.8%	#of variable bits:55 Avalanche%: 50.9%	Changing just one bit in the seed resulted in a change of approximately 50% of the resulting keystream, demonstrating the strong Avalanche Effect. This means that even a small change in the key causes a significant change in the keystream, making the key harder to predict and improving encryption security.

Weaknesses and Limitations

1. Baseline LFSR

In the baseline version, the keystream is generated relatively long, so the system needs more time before iteration begins, which is a strength. However, because the LFSR used in the baseline relies on a simple, linear relationship for bit generation, the encryption pattern can be easier to predict or analyze. Furthermore, if the same seed is used repeatedly, the system will generate the same keystream again, potentially allowing an attacker to compare the encrypted messages and extract information from the plain text. Also, if the plain text is very long relative to the cycle length, the keystream might start iterating during encryption, reducing the security level and increasing the likelihood of pattern detection within the cipher text. Therefore, the system is more vulnerable to being broken.

2. Enhancement LFSR

In the enhancement version, the keystream generation process is made more complex by adding non-linear elements, making the relationship harder to predict and thus increasing security compared to the baseline. However, the number of rounds in the enhancement is less than in the baseline, meaning the cycle length is shorter. Therefore, if the source text is long, the probability of the keystream being duplicated within the same message increases, potentially making the system more vulnerable to pattern detection or relationship analysis between the source and cipher text. Furthermore, using the same seed multiple times generates the same keystream again, a significant limitation as it could allow an attacker to compare cipher texts and extract information from the source text. Thus, while the enhancement offers greater complexity and security, it still needs to prevent seed reuse and ensure keystream duplication, especially with long text strings.

Analysis of resistance to basic attacks

The encryption system in this project is based on the Stream Cipher concept, using a keystream generator built on LFSR. Generally, this type of system is fast and efficient, but it can be vulnerable to attacks if randomness and complexity in the generation process are not enhanced. The Baseline version relies on a simple linear structure, making the relationship between the seed and the

keystream more amenable to mathematical analysis and therefore less resistant to attacks targeting linear systems. The Enhancement version has been improved by introducing nonlinear operations and increasing the complexity of keystream generation. This helps reduce the predictability of the output and makes the system more difficult to break using linear analysis or key deduction. However, key management remains crucial, as reusing the same seed or generating the same keystream repeatedly can compromise security.

Attack Scenarios

One of the most significant weaknesses in Stream Cipher algorithms like LFSR is the problem of reusing the same seed or keystream multiple times. If the same keystream is generated to encrypt two different messages, an attacker can perform XOR between the two cipher texts and negate the effect of the keystream, revealing a direct relationship between the original texts. This could allow them to extract parts of the plaintext or even deduce the keystream itself with additional information. Therefore, reusing the same key or seed is a major risk because it reduces the security level and makes the system vulnerable to attacks such as Keystream Reuse Attacks and Known Plaintext Attacks. For this reason, it is essential to use a different seed or a new keystream for each encryption operation to ensure better protection.

1. Keystream Reuse Attack

```
Plaintext 1: HELLO
Plaintext 2: WORLD


------------Baseline key-stream reuse attack--------------------

Baseline Ciphertext 1 (binary): 1111110100100111101010010000010011010110
Baseline Ciphertext 2 (binary): 1110001000101101101101110000010011011101
Baseline XOR of two ciphertexts: 0b1111100001010000111100000000000001011


------------Enhancement key-stream reuse attack--------------------

Enhancement Ciphertext 1 (binary): 1111110010110000110101010111000100100101
Enhancement Ciphertext 2 (binary): 1110001110111010110010110111000100101110
Enhancement XOR of two ciphertexts: 0b1111100001010000111100000000000001011
```

Figure 11: result Keystream Reuse Attack

Applied a keystream reuse attack to both Baseline and Enhancement encryption systems by encrypting two different plain texts (HELLO and WORLD) using the same keystream each time. We observed that the cipher text values differed between the two systems due to the different keystream generation methods. However, the result of the XOR operation between the first and second cipher texts was identical in both cases. This is because using the same keystream results in its cancellation during the XOR operation, making the result dependent solely on the original texts and not on the encryption method used. Furthermore, this attack demonstrates that an attacker can access the original texts through analysis and experimentation. Obtaining an XOR between the two encrypted texts directly yields an XOR between the original texts.

C1 XOR C2 = (P1 XOR K) XOR (P2 XOR K) => K XOR K = 0, X XOR 0 = X

C1 XOR C2 = P1 XOR P2

By using logical reasoning or predicted words, parts of the original text can be inferred and then recovered in whole or in part. This shows that keystream reuse poses a serious security vulnerability, even in enhanced systems.

As a result of this attack, sensitive information such as the relationship between original texts, character patterns, and message lengths can be leaked, in addition to revealing parts of the original text or even recovering it entirely if a portion of the text is predictable or partially known. The more predictable the linguistic structure of the messages, the more information can be leaked.

2. Known Plaintext Attack

```
Plaintext 1: HELLO
Plaintext 2: WORLD


------------Baseline known plaintext attack--------------------

Baseline known part keystream (binary): 0b1011010101100010
plain_part as ASCII: WO


------------Enhancement known plaintext attack--------------------

Enhancement known part keystream (binary): 0b10110100111110101
plain_partenhancement as ASCII: WO
```

Figure 12: result known plaintext attack

Let's assume the attacker knows a portion of the plaintext and its corresponding cipher text. With this knowledge, the attacker can perform an XOR operation between the known portion of the plaintext and the cipher text, allowing them to extract a portion of the keystream used in encryption. As shown in the results, we were able to recover a portion of the plaintext (such as "WO") because the same keystream (seed) was reused across messages. This means that knowing a small part of the plaintext was sufficient to reveal parts of the rest of the message. In the case of the baseline, the relationship between the keystream and the state of the registers is relatively simple, making it easier for the attacker to analyze this relationship and attempt to predict or retrieve the key. In the enhancement system, the relationship is more complex due to the use of nonlinear operations, making analysis more difficult. However, this complexity is not sufficient protection, as keystream reuse remains a fundamental vulnerability. The attacker can still guess the remaining bits or exploit other known portions of the text to obtain additional information about the message.

In implementing the attack scenarios in this project, it was assumed that the attacker possessed only the cipher text without knowing the value of the seed. This assumption helps in analyzing the system's strength in the absence of additional information for the attacker. However, in the actual encryption design, the seed is transmitted as part of the cipher text itself, meaning that an attacker can obtain it if they discover its location within the message. In some cases, the presence of a seed within the cipher text may initially cause some confusion for the attacker, especially if it is not clear that a portion of the data is the seed and not part of the cipher text. However, if the attacker is aware of this, keystream retrieval becomes easier, particularly in the Baseline version due to the linear nature of the generation. In the Enhancement version, however, the added complexity of keystream generation makes analysis relatively more difficult, even with the seed available.